

A (More) Objective View of AI

SOFTWARE DEVELOPMENT IN THE AGE OF AI

Federico Busato

2026-06-20

1. Software Employment

2. Limitations of AI-Generated Code

2.1 The Slot Machine and the Illusion of Control

2.2 Generalization

2.3 Creativity

2.4 The Illusion of Competence

2.5 Long-Term Tasks

2.6 The “Last Mile” Problem

3. Implications of AI-Generated Code

3.1 Productivity

3.2 Deskilling

3.3 AI Addiction

3.4 Technical Debt

3.5 Security Vulnerabilities

3.6 Maintainability

3.7 Code Review

3.8 Comprehension Debt

3.9 Cost

4. **AI as a Software Engineering Tool**

4.1 Code Generation is NOT Software Engineering

4.2 The Role of Human Expertise

5. **Engineering Practices in the Age of AI**

1. Software Employment

Despite the daily dystopian and gloomy predictions on social media and news sites, AI will not replace software developers. In fact, demand for software developers is and will remain strong.

Layoffs and the decline in entry-level job postings have been driven by **massive capital spending on infrastructure** and economic uncertainty, disproportionately affecting early-career roles. *The industry is effectively trading junior headcount for compute capacity.* ^{1, 2, 3}

¹ [Why AI Companies May Invest More than \\$500 Billion in 2026](#) ↗ - GOLDMAN SACHS

² [AI isn't replacing jobs. AI spending is](#) ↗ - FAST COMPANY, 2025

³ [The cost of compute: A \\$7 trillion race to scale data centers](#) ↗ - MCKINSEY & COMPANY, 2025

A Harvard Business School study ⁴ found that at firms adopting AI, junior employment drops significantly while senior employment remains flat. The researchers warn that this is a “*defensive move*” that boosts short-term efficiency but **risks a long-term talent crisis**, as fewer human experts will be trained to supervise the AI systems of the future.

⁴ [Generative AI as Seniority-Biased Technological Change](#) ↗ - SOCIAL SCIENCE RESEARCH NETWORK, 2025 6 / 81

*“The impact of productivity gains on jobs depends on whether the demand for goods produced by that work is **elastic** or **inelastic**. If demand is inelastic, productivity gains mean jobs will be lost...*

... If product demand is sufficiently elastic, productivity-enhancing technology will increase industry employment”⁵

— **Shaping AI's Impact on Billions of Lives**

⁵ Shaping AI's Impact on Billions of Lives  - ARXIV, 2025 / COMMUNICATIONS OF THE ACM, 2026

Software development has elastic demand. It offers great potential and opportunities, driven by its integration into every aspect of life and business and by rapid technological evolution.

- The US Bureau of Labor Statistics predicts a **25% increase** in software developers over the next decade ⁶.
- The World Economic Forum lists software and application developers among the **fastest-growing jobs** in the 2025-2030 timeframe ⁷.
- The global software market size is projected to reach \$1.4 trillion by 2030, growing at a compound annual **growth rate** (CAGR) of **+11.3%** ⁸.

⁶ Employment Projections, 2025-2035  - US BUREAU OF LABOR STATISTICS, 2025

⁷ Future of Jobs Report, 2025-2030  - WORLD ECONOMIC FORUM, 2025

⁸ Software Market (2025 - 2030)  - GRAND VIEW RESEARCH, 2025

- Engineering headcount changes since 2022: Meta **+19%**, Google **+16%**, Apple **+13%** ⁹.

~ Jobs with high exposure to AI have seen greater wage and stronger job growth than jobs minimally exposed to AI ¹⁰.

— **AI exuberance: Economic upside, stock market downside**

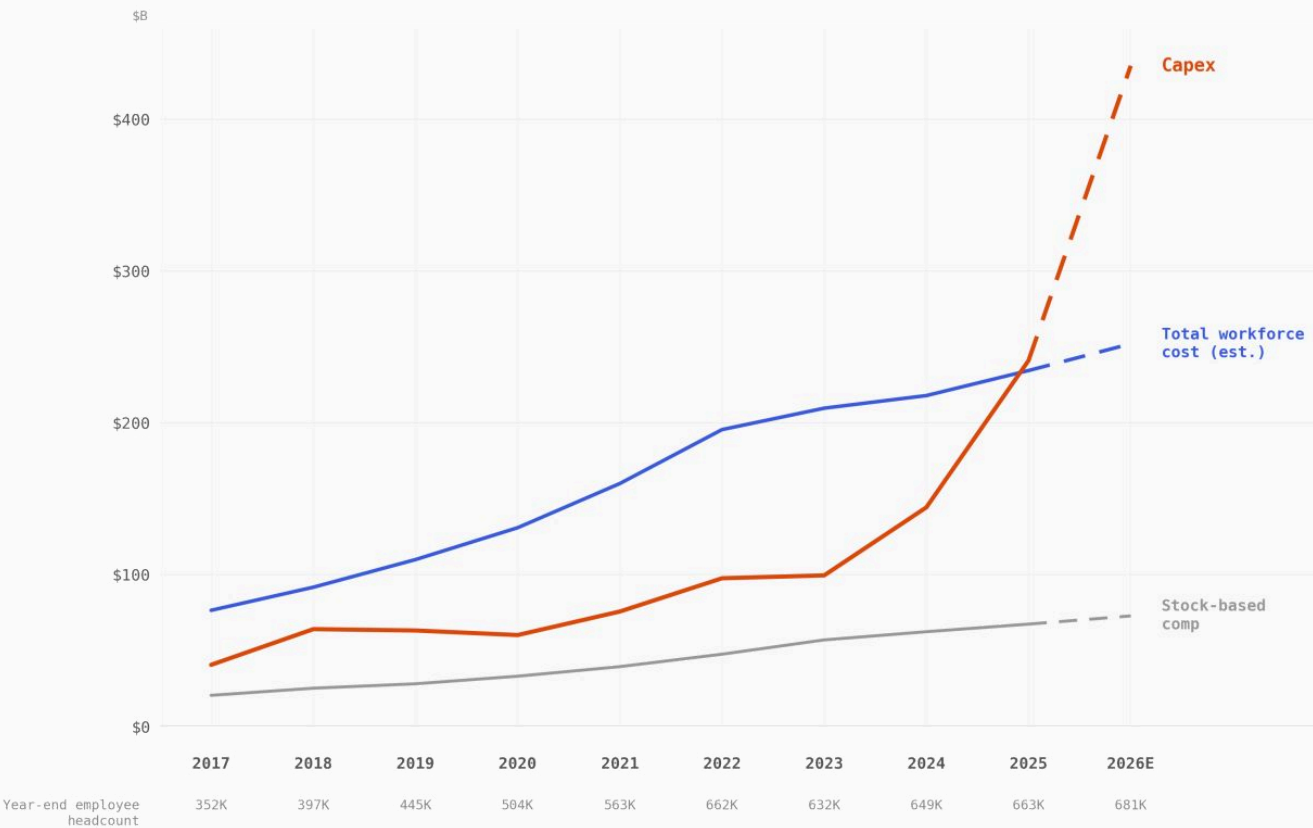
⁹ Software Engineer Job Market 2025  - UNDERDOG.IO, 2025

¹⁰ AI exuberance: Economic upside, stock market downside  - VANGUARD ECONOMIC AND MARKET OUTLOOK FOR 2026

ALPHABET + META + APPLE + MICROSOFT · FY 2017–2026E

When Capex Surpasses People

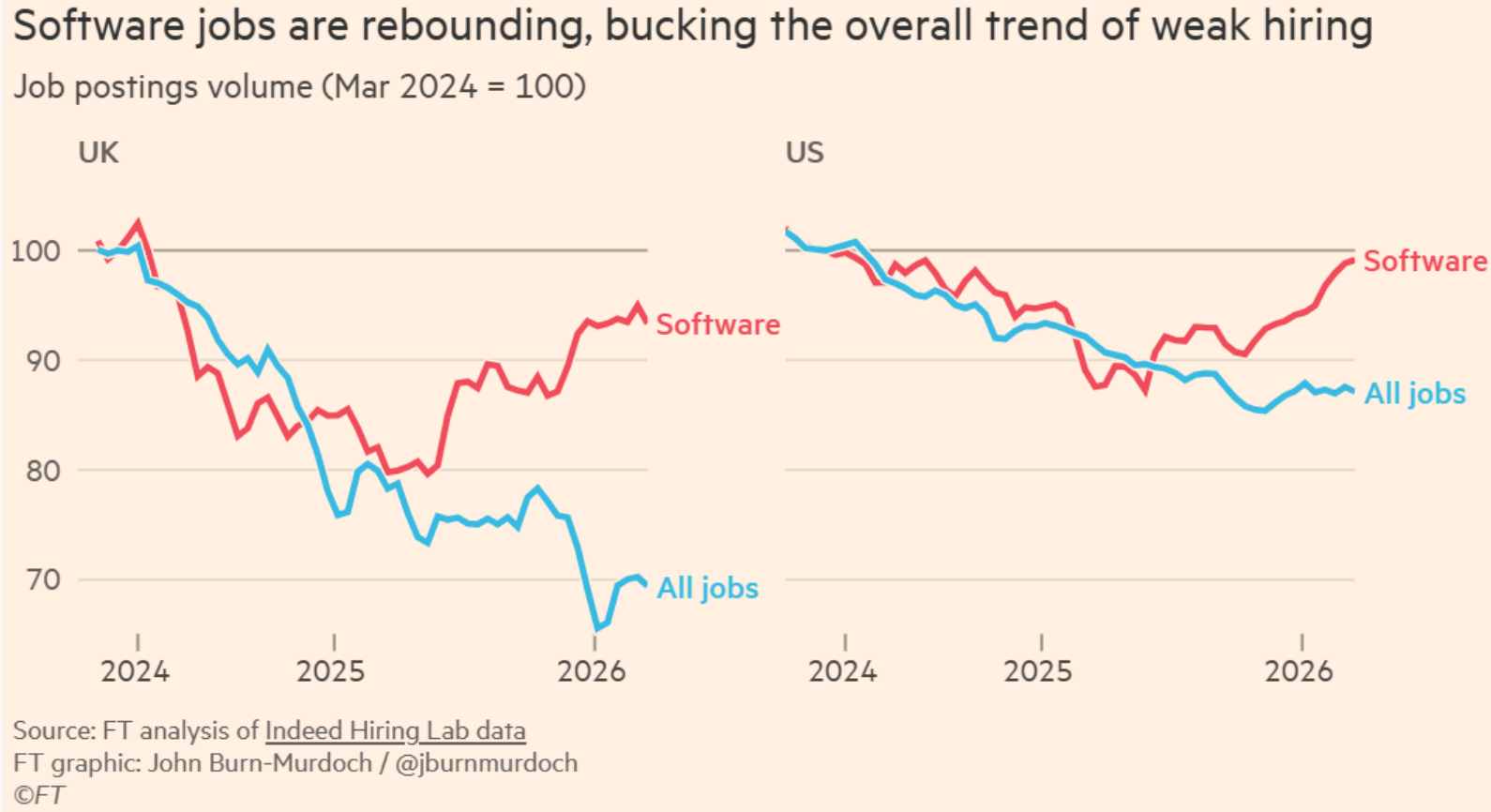
Capital expenditure vs. total workforce cost



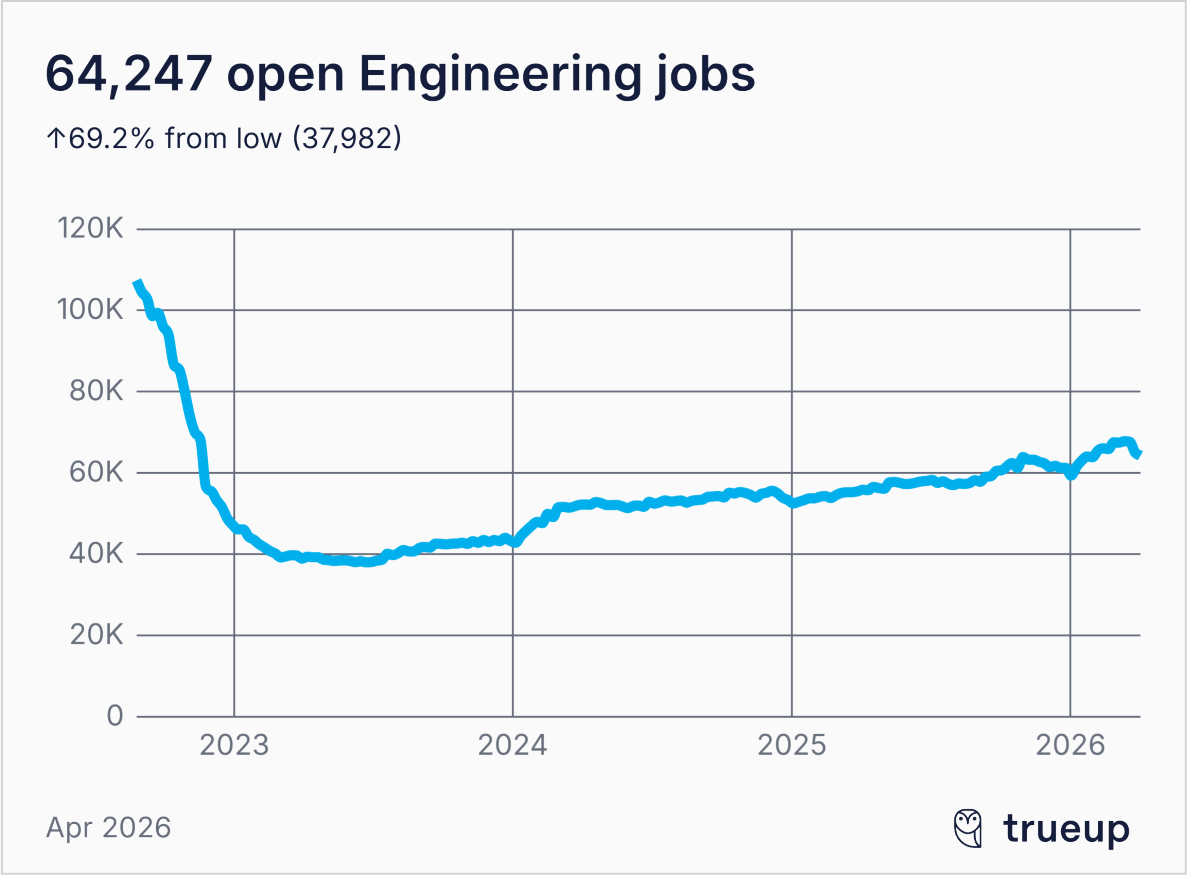
Occupations with the highest projected total change in employment, 2024-2034 ¹²

Occupation	Projected new jobs	Median annual pay, 2024
Home health and personal care aides	739.8K	\$34.9K
Software developers	267.7K	\$133.1K
Stockers and order fillers	235K	\$37.1K
Fast food and counter workers	233.2K	\$30.5K
Cooks, restaurant	217K	\$36.8K
Registered nurses	166.1K	\$93.6K
General and operations managers	164K	\$103K
Medical and health services managers	142.9K	\$118K
Financial managers	128.8K	\$161.7K
Nurse practitioners	128.4K	\$129.2K
Construction laborers	106.5K	\$46.7K
Computer and information systems managers	101.6K	\$171.2K

¹² What are the fastest-growing professions in America?  - USAFACTS, 2025



The AI Shift: Will software engineers survive agentic AI?  - SARAH & MURDOCH, FINANCIAL TIMES, 2026



“Without the hiring of early-in-career developers, the profession’s talent pipeline will collapse, and organizations will face a future without the next generation of experienced engineers.” ¹⁵

— **Redefining the Software Engineering Profession for AI**,
COMMUNICATIONS OF THE ACM, 2026

¹⁵ Redefining the Software Engineering Profession for AI  - RUSSINOVICH & HANSELMAN,
COMMUNICATIONS OF THE ACM, 2026

“Every organization cutting engineering headcount on the basis of AI output gains should read this report. The AI engineering impact data shows that output is up. It also shows that the work required to ensure that output is safe, correct, and maintainable has not decreased. It has increased substantially.”¹⁶

— **Ten takeaways from the AI Engineering Report 2026:
The Acceleration Whiplash,**
FAROS RESEARCH, 2026

¹⁶ Ten takeaways from the AI Engineering Report 2026: The Acceleration Whiplash  - FAROS RESEARCH, APRIL 2026

*“Now that we have software that can write working code, **what is there left for us humans to do?***

The answer is so much stuff.

Writing code has never been the sole activity of a software engineer. The craft has always been figuring out what code to write. Any given software problem has dozens of potential solutions, each with their own tradeoffs. Our job is to navigate those options and find the ones that are the best fit for our unique set of circumstances and requirements.”¹⁷

— **What is agentic engineering?**,
S. WILLISON, 2026

¹⁷ [What is agentic engineering?](#) ↗ - S. WILLISON, 2026

2. Limitations of AI-Generated Code

The Slot Machine and the Illusion of Control

*“The thing about AI based coding is that it’s like a **slot machine**, and that you have an **illusion of control**, you know, you can get to craft your prompt, and your list of MCPs, and your skills, and whatever, and then in the end, you pull the lever. Right?*

... It’s the stochastic thing. You get the occasional win. It’s like, oh, I won. I got a feature.”¹⁸

— The Dangerous Illusion of AI Coding?,
JEREMY HOWARD, 2026

¹⁸ [The Dangerous Illusion of AI Coding?](#)  - JEREMY HOWARD, 2026

- Lack of generalization beyond the training data.

*“This brittleness means that **coding agents are almost frighteningly good at what they’ve been trained and fine-tuned on—modern programming languages, JavaScript, HTML, and similar well-represented technologies—and generally terrible at tasks on which they have not been deeply trained.***

... It took me five minutes to make a nice HTML5 demo with Claude but a week of torturous trial and error, plus actual systematic design on my part, to make a similar demo of an Atari 800 game.”⁴⁹

— Benj Edwards,
ARS TECHNICA

*“Due to what might poetically be called “preconceived notions” baked into a coding model’s neural network (more technically, statistical semantic associations), **it can be difficult to get AI agents to create truly novel things**, even if you carefully spell out what you want.”* ⁴⁹

— Benj Edwards,
ARS TECHNICA

- Bug proliferation

*“Fixing bugs can also create bugs elsewhere. Coding agents **supercharge this phenomenon** because they can barrel through your code and make sweeping changes in pursuit of narrow-minded goals that affect lots of working systems.”* ⁴⁹

— Benj Edwards, ARS TECHNICA

*“A comprehensive report analyzed 470 real-world open-source pull requests and found that **AI-generated code introduces 1.7x more defects** across every major category of software quality—including logic, maintainability, security, and performance.”* ¹⁹

— CodeRabbit

¹⁹ State of the AI vs. Human Code Generation Report  - CODERABBIT, 2025

“LLMs cosplay understanding things.

*... The difference between pretending to be intelligent and actually being intelligent is entirely unimportant, as long as you're in the region in which the pretense is actually effective, you know. So it's actually fine for a great many tasks that LLMs only pretend to be intelligent, because for all intents and purposes, it just doesn't matter **until you get to the point where it can't pretend anymore**. And then you realize, like, oh my god. This thing is so stupid.”¹⁸*

— Jeremy Howard,

THE DANGEROUS ILLUSION OF AI CODING?, 2026

*“61% agree that “AI often produces code that looks correct but isn’t reliable.” That’s a critical finding—it means **AI code can introduce subtle bugs that are harder to spot than typical human errors.***

*The same percentage (61%) agree that it “**requires a lot of effort to get good code from AI**” through prompting and fixing.”²⁰*

— Sonar,
STATE OF CODE DEVELOPER SURVEY REPORT, 2026

²⁰ [State of Code Developer Survey report](#)  - SONAR, 2026

“Overall performance scores drop significantly from $> 80\%$ on isolated tasks to at most 38% in continuous settings, exposing agents’ profound struggle with longterm maintenance and error propagation.”²¹

— **EvoClaw: Evaluating AI Agents on Continuous Software Evolution**,
DENG ET AL., 2026

²¹ [EvoClaw: Evaluating AI Agents on Continuous Software Evolution](#)  - DENG ET AL., ARXIV, JUNE 2026

1. **Context drift.** *As codebases grow beyond the effective context window, agents lose coherent understanding of system-wide invariants and dependencies.*
2. **Error propagation.** *A small error in an early commit cascades into compounding failures in subsequent work, and agents lack robust mechanisms for detecting and recovering from these chains.*
3. **Technical debt awareness.** *Agents do not currently model the long-term costs of their design decisions — they optimize for immediate task completion without considering maintainability.*
4. **Verification fidelity.** *Automated testing remains incomplete; agents can pass tests while introducing subtle semantic errors that only manifest under novel inputs.* ²²

²² [Agentic Software: How AI Agents Are Restructuring the Software Paradigm](#)  - Z. CAO, ARXIV, JUNE 2026

— **Agentic Software: How AI Agents Are Restructuring the Software Paradigm,**
Z. CAO, 2026

- **The “Last Mile” Problem.** Prototyping with AI is not the same as building a production-quality product.

*“The first 90 percent of an AI coding project comes in fast and amazes you. **The last 10 percent involves tediously filling in the details** through back-and-forth trial-and-error conversation with the agent.” ⁴⁹*

— **Benj Edwards,**
ARS TECHNICA, 2026

“Honest reflections from coding with AI so far as a non-engineer:

*It can get you 70% of the way there, but that **last 30% is frustrating**. It keeps taking one step forward and two steps backward with new bugs, issues, etc.*

*If I knew how the code worked I could probably fix it myself. But since I don't, **I question if I'm actually learning that much.**”²³*

— Peter Yang

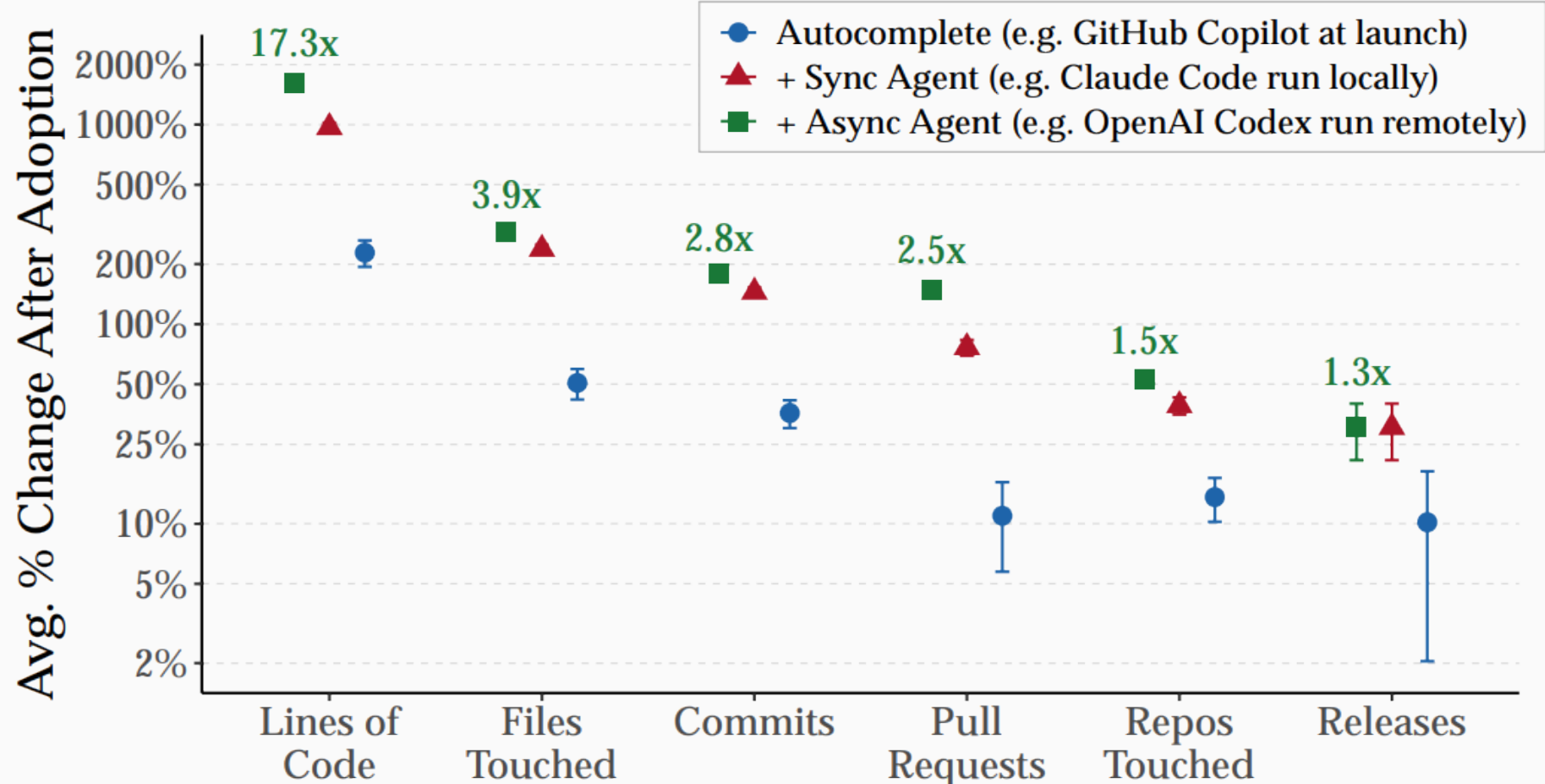
²³ [Peter Yang](#) 

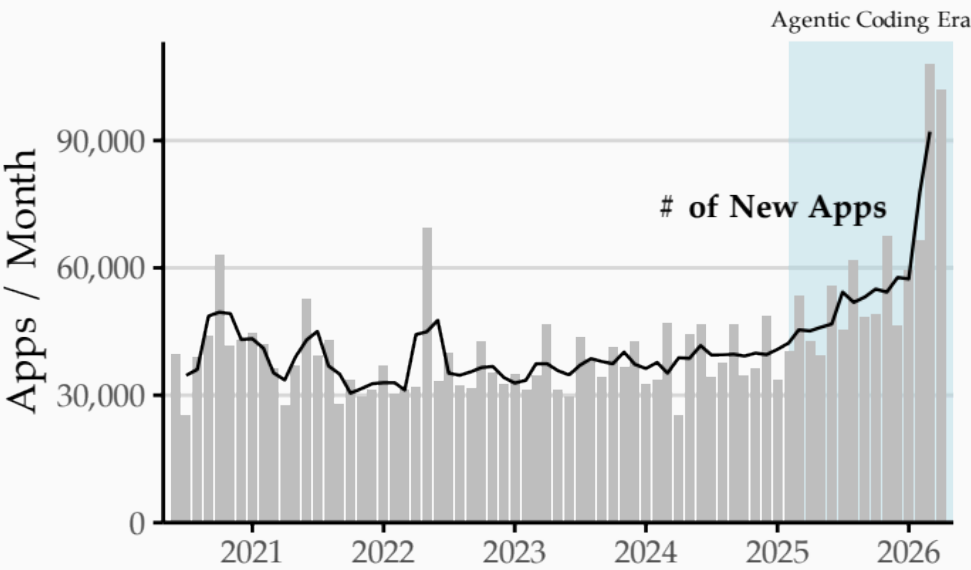
With AI, it's easier to get the first 90 percent out there. This means we can spend more time on the remaining 10 percent, which means more time for craftsmanship and figuring out how to make your users happy. ²⁴

— **The 100 hour gap between a vibecoded prototype and a working product,**
M. BUDKOWSKI, 2026

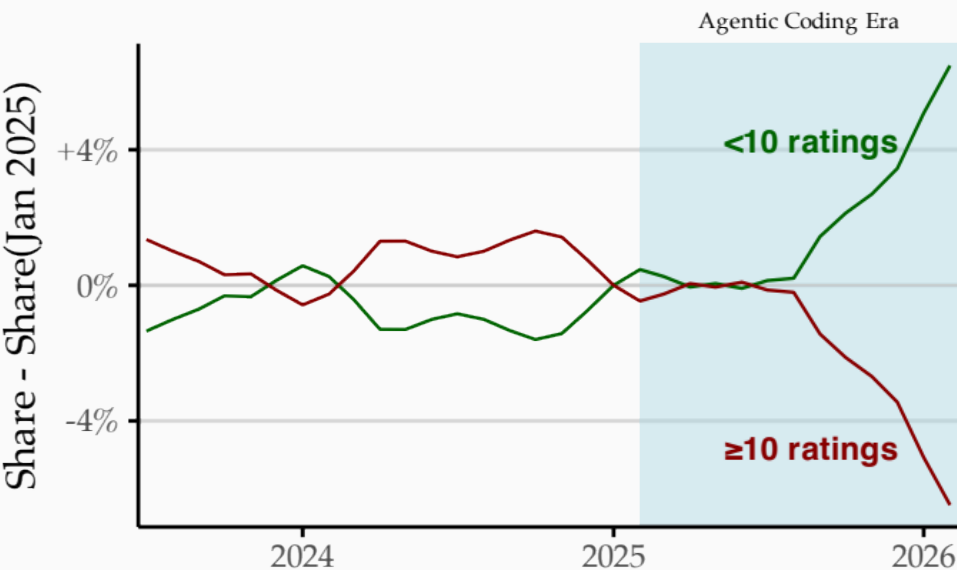
²⁴ [The 100 hour gap between a vibecoded prototype and a working product](#)  - M. BUDKOWSKI, 2026

3. Implications of AI-Generated Code





(a) iOS: New Apps



(g) iOS: New Apps by Usage Bin

*“AI coding tools produce increasingly large task-level productivity effects. Yet these gains attenuate sharply across the production hierarchy: **sync agents lead to a 741% increase in lines of code and a 65% increase in pull requests, but releases rise by only 20%.***

*... Across four major marketplaces, **new application creation has accelerated** since mid-2025, but total usage within the first three months has not risen, and **the share of new applications that fail to reach even a modest audience has increased.**“ ²⁵*

— **Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools,**

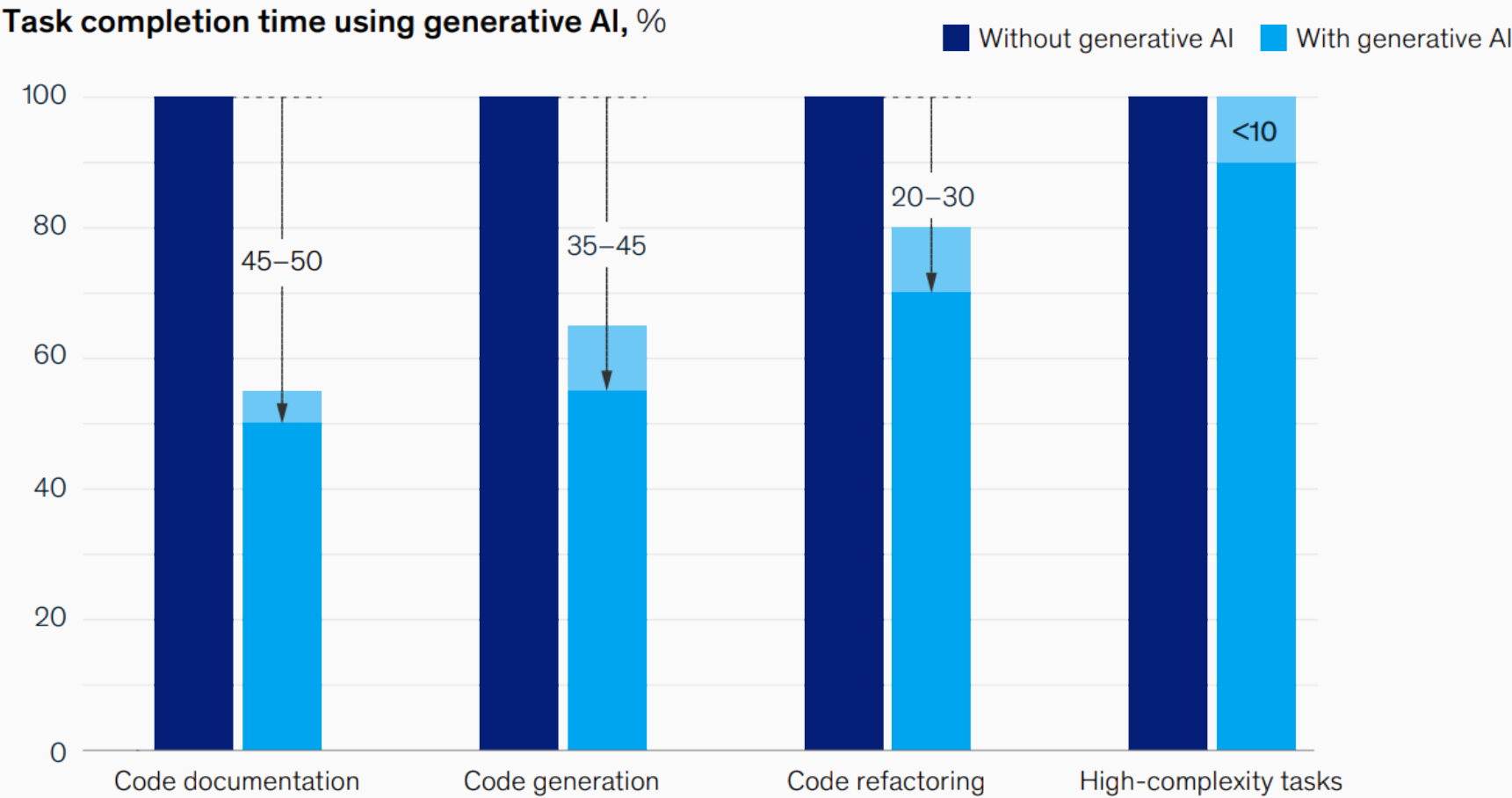
NATIONAL BUREAU OF ECONOMIC RESEARCH, 2026

²⁵ [Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools](#)  - DEMIRER ET AL., NATIONAL BUREAU OF ECONOMIC RESEARCH, 2026

“Epics completed per developer are up 66%. Task throughput per developer is up 33.7%. PR merge rate per developer is up 16.2%. These numbers represent real delivery acceleration...

But the throughput numbers have an asterisk... “¹⁶

— **Ten takeaways from the AI Engineering Report 2026:
The Acceleration Whiplash,**
FAROS RESEARCH, 2026



“People who go all in on AI agents now are guaranteeing their obsolescence. If you outsource all your thinking to computers, you stop upskilling, learning, and becoming more competent.”²⁷

— 7 Questions with Jeremy Howard on Open Source AI and Agents,
JEREMY HOWARD, 2026

²⁷ 7 Questions with Jeremy Howard (Answer.ai, fast.ai) on Open Source AI and Agents [↗](#) - JEREMY HOWARD, 2026

“incorporating AI aggressively into the workplace, particularly with respect to software engineering, comes with trade-offs.

*Given time constraints and organizational pressures, junior developers or other professionals may **rely on AI to complete tasks as fast as possible at the cost of skill development** — and notably the ability to debug issues when something goes wrong.”*²⁸

— **How AI assistance impacts the formation of coding skills,**
ANTHROPIC, 2026

²⁸ [How AI assistance impacts the formation of coding skills](#) ↗ - ANTHROPIC, 2026

*“When I was browsing LinkedIn during the Claude outage that occurred a bit ago, I noticed numerous posts highlighting that certain developers and engineering teams were at a standstill. Their workflows, their own coding abilities, had already reached a point where they were largely dependent on these vendors. **What used to be a skill that they could execute with just a keyboard and text editor suddenly required a subscription to an AI model provider.**”* ²⁹

— **Agentic Coding is a Trap,**
LARS FAYE, 2026

²⁹ [Agentic Coding is a Trap](#) ↗ - LARS FAYE, 2026

- **AI-assisted code has contributed to a measurable decline in code quality.** Code generated by LLMs without engineering supervision tends to deteriorate quickly over time due to duplicated logic, inconsistent naming, and poor architectural organization. This translates into rising maintenance costs, lower developer satisfaction, and fragile systems.

My project became so big that claude can't properly understand it

Question

So, I made a project in python entirely using Cursor (composer) and Claude, but it has gotten to a point that the whole codebase is over 30 Python files, code is super disorganized, might even have duplicate loops, and Claude keeps forgetting basic stuff like imports at this point. When I ask it to optimize the code or to fix a bug, it doesn't even recognize the main issue and just ends up deleting random lines or breaking everything completely.

I have 0 knowledge about python, it's actually a

*“**Code churn**, the ratio of lines deleted to lines added for merged code in a given quarter, **has increased 861% under high AI adoption**. At nearly 10 times the prior rate, **significantly more code is being removed relative to what is being added**.*

***The incidents-to-PR ratio is up 242.7%** as teams move from low to high AI adoption. ... For every PR merged, incidents are occurring at more than three times the rate relative to the low AI adoption baseline.*

***bugs per developer has risen to 54%**. ... More AI-generated code in the codebase correlates with more bugs per developer, and that relationship is strengthening as adoption deepens.”¹⁶*

— Ten takeaways from the **AI Engineering Report 2026:**
The Acceleration Whiplash,
FAROS RESEARCH, 2026

*“**The share of copy/pasted lines surged** from 8.3% in 2020 to 12.3% in 2024, a 48% relative increase.*

*The proportion of new code that was revised within two weeks of its initial commit grew from 3.1% in 2020 to 5.7% in 2024, indicating a **rise in premature or low-quality commits**.*

*The percentage of revised code that was originally written more than a month prior dropped from 30% in 2020 to just 20% in 2024. This suggests that **developers are spending more time modifying recently written AI-generated code rather than improving or refactoring legacy systems.**”³⁰*

— GitClear,
AI COPILOT CODE QUALITY, 2025

³⁰ [AI Copilot Code Quality: 2025 Look Back at 12 Months of Data](#)  - GITCLEAR, 2025

*“Relative code churn (refactoring) far outpaces increases in productive output. Regular AI users averaged 9.4x higher code churn than their non-AI counterparts, **which is 2.2x greater than their composite productivity increase.***

- *Code Review Minutes 3.1x*
- *Copy/Paste Lines 4.1x*
- *Churn Lines 9.4x”* ³¹

— GitClear,

AI CODING TOOLS ATTRACT TOP PERFORMERS — BUT DO THEY CREATE THEM?, 2026

³¹ [AI Coding Tools Attract Top Performers - But Do They Create Them? ↗ - GITCLEAR, 2026](#)

*“In the 1980 Turing Award lecture Tony Hoare said: ‘There are two ways of constructing a software design: one way is to make it so simple that there are obviously no deficiencies, and the other is to make it so complicated that there are no obvious deficiencies.’ This LLM-generated code falls into the second category. The reimplementation is 576,000 lines of Rust. **That is 3.7x more code** than SQLite.”* ⁵³

— **Your LLM Doesn't Write Correct Code. It Writes Plausible Code,**
HŌRŌSHI, 2026

“With AI coding agents, do you know you can accumulate years of technical debt in a matter of days.

How do you prevent your codebase from becoming unmaintainable? Remember, these coding agents often rely heavily on text-search patterns to understand and modify code.”³²

— Sung Kim, 2026

³²  - SUNG KIM, 2026

- **AI-generated code can introduce real risks for security and reliability.**

*“We produce 89 different scenarios for Copilot to complete, producing 1,689 programs. Of these, **we found approximately 40% to be vulnerable.**”*³³

— **Asleep at the Keyboard?**, PEARCE ET AL.

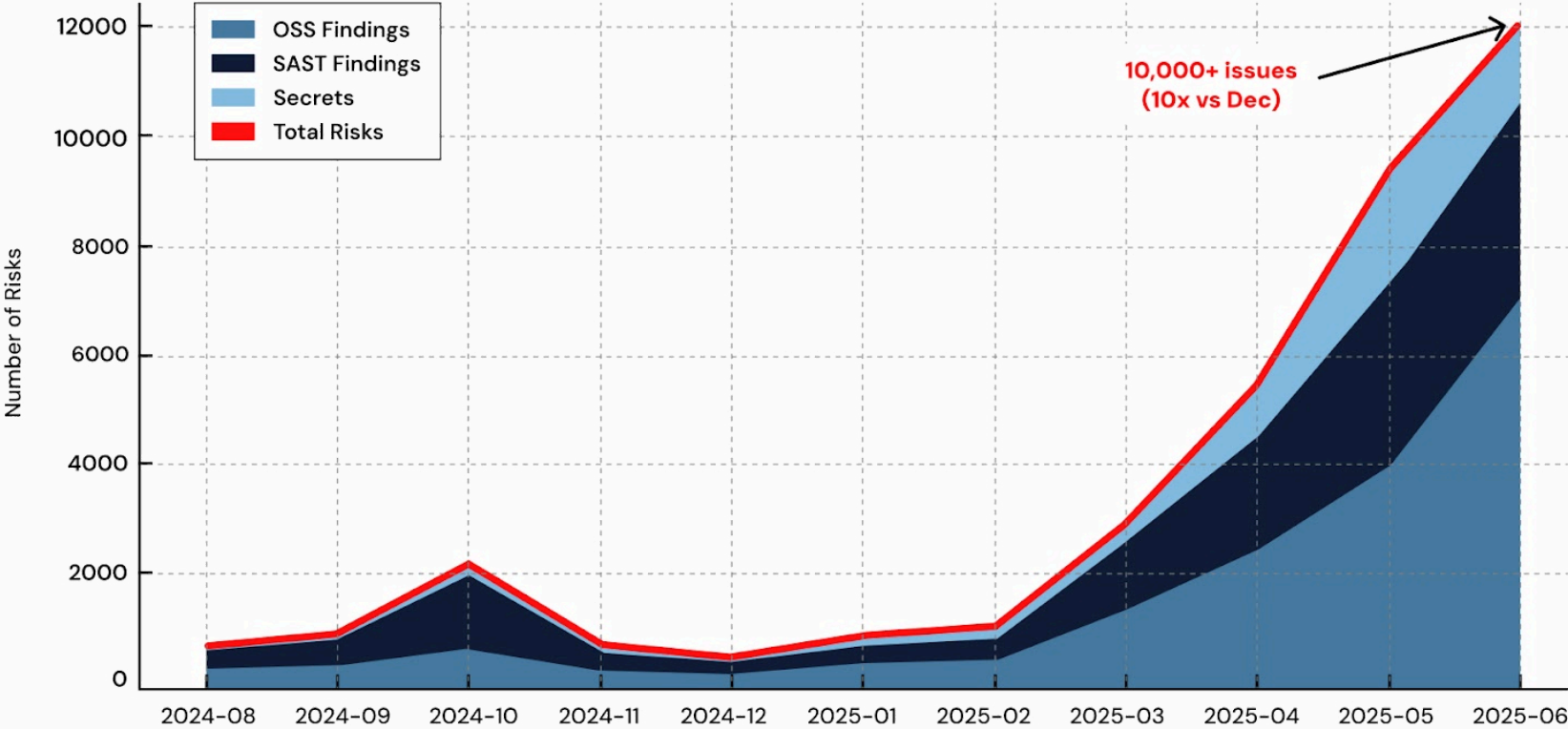
*By June 2025, AI-generated code was introducing over 10,000 new security findings per month across the repositories in our study—a 10x spike in just six months compared to December 2024. And **the curve isn’t flattening; it’s accelerating.***³⁴

— **4x Velocity, 10x Vulnerabilities**, APIIRO

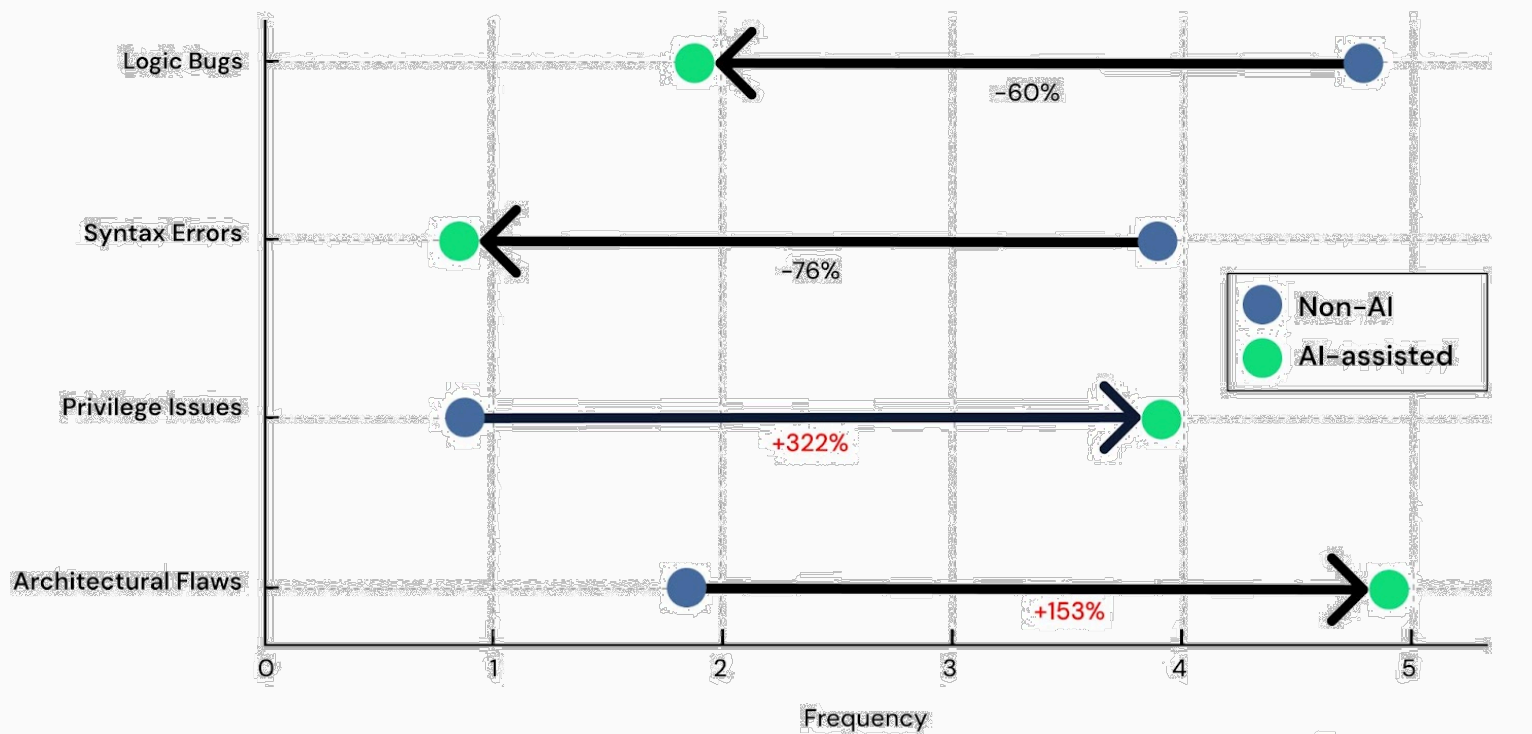
³³ [Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions](#)  - PEARCE ET AL., COMMUNICATIONS OF THE ACM, 2025

³⁴ [4x Velocity, 10x Vulnerabilities: AI Coding Assistants Are Shipping More Risks](#)  - APIIRO, 2025

AI Code Pushed 10,000+ New Issues in a Single Month – a 10x Spike in 6 Months



AI Suppresses Shallow Bugs, Amplifies Deep Flaws
Relative change from Non-AI to AI-assisted (1= rare, 5= common)



*“We are still early, and **fully autonomous development comes with real risks**. When a human sits with Claude during development, they can ensure consistent quality and catch errors in real time. For autonomous systems, **it is easy to see tests pass and assume the job is done, when this is rarely the case.**” ³⁵*

— Building a C compiler with a team of parallel Claudes,
ANTHROPIC, 2026

³⁵ [Building a C compiler with a team of parallel Claudes](#)  - ANTHROPIC, 2026

“AI coding tools have caused as many problems as they have solved, according to industry experts. The easy-to-use and accessible nature of AI coding tools has enabled a flood of bad code that threatens to overwhelm projects. Building new features is easier than ever, but maintaining them is just as hard and threatens to further fragment software ecosystems.”

... Blender Foundation CEO Francesco Siddi said LLM-assisted contributions typically ‘wasted reviewers’ time and affected their motivation.

... The open source data transfer program cURL recently halted its bug bounty program after being overwhelmed by what creator Daniel Stenberg described as ‘AI slop.’”³⁶

— For open source programs, AI coding tools are a mixed blessing,
TECHCRUNCH, 2026

³⁶ For open source programs, AI coding tools are a mixed blessing [↗](#) - R. BRANDOM, TECHCRUNCH, 2026

“70% of engineering leaders say maintaining their test suite is now a bigger burden than writing code itself

81% of organizations have seen production issues increase linked to AI-generated code, “³⁷

— **The State of Code Abundance 2026**,
CLOUDBEES, 2026

³⁷ [The State of Code Abundance 2026](#)  - CLOUDBEES, 2026

“70% of engineering leaders say maintaining their test suite is now a bigger burden than writing code itself

81% of organizations have seen production issues increase linked to AI-generated code, “³⁸

— **The State of Code Abundance 2026**,
CLOUDBEES, 2026

³⁸ [The State of Code Abundance 2026](#)  - CLOUDBEES, 2026

“The most experienced people in your organization are being buried...AI-generated code presents a specific and under-appreciated challenge for reviewers.

*Median time to first PR review is up 156.6%. Average time spent in code review is up 199.6%. Median time in review is up 441.5%. **The engineers with the deepest knowledge of the system are spending their most valuable hours unraveling plausible-looking code that should never have reached them in the state it did.**” ¹⁶*

— Ten takeaways from the AI Engineering Report 2026:
The Acceleration Whiplash,
FAROS RESEARCH, 2026

The traditional loop — request review, wait for code owner, merge — breaks down when one developer can kick off a dozen agent sessions before lunch.
Throughput has scaled exponentially. Human review capacity hasn't. The gap is widening.

*Reviewing your own pull request isn't optional when agents are involved. **It's basic respect for your reviewer's time.***³⁹

— Agent pull requests are everywhere. Here's how to review them,
GITHUB, 2026

³⁹ [Agent pull requests are everywhere. Here's how to review them](#)  - GITHUB, 2026

“Comprehension debt is the hidden cost to human intelligence and memory resulting from excessive reliance on AI and automation. For engineers, it applies most to agentic engineering.

There’s a cost that doesn’t show up in your velocity metrics when teams go deep on AI coding tools. Especially when its tedious to review all the code the AI generates. This cost accumulates steadily, and eventually it has to be paid - with interest.”

“Unlike technical debt, which announces itself through mounting friction - slow builds, tangled dependencies, the creeping dread every time you touch that one module - **comprehension debt breeds false confidence**. The codebase looks clean. The tests are green. **The reckoning arrives quietly, usually at the worst possible moment.**

... no one on the team could explain why design decisions had been made or how different parts of the system were supposed to work together. The theory of the system had evaporated.

That’s comprehension debt compounding in real time.”

“AI-generated code breaks that feedback loop. The volume is too high. The output is syntactically clean, often well-formatted, superficially correct - precisely the signals that historically triggered merge confidence. **But surface correctness is not systemic correctness.**

*... Research suggests that developers using AI for code generation delegation score below 40% on comprehension tests, while developers using AI for conceptual inquiry - asking questions, exploring tradeoffs - score above 65%. **The tool doesn't destroy understanding. How you use it does.***

*... **As AI volume goes up, the engineer who truly understands the system becomes more valuable, not less.**“ ⁴⁰*

— Comprehension Debt - The Hidden Cost of AI Generated Code,
A. OSMANI, 2026

⁴⁰ Comprehension Debt - The Hidden Cost of AI Generated Code  - A. OSMANI, 2026

“54% report a significant increase in CI/CD infrastructure spend in the past 12 months. 53% report a significant increase in testing, security, and deployment costs.

Less than half (45%) of teams describe their AI spend as highly predictable quarter-to-quarter. 27% have set hard limits or quotas on token usage”⁴¹

— **The State of Code Abundance 2026**,
CLOUDBEES, 2026

⁴¹ [The State of Code Abundance 2026](#)  - CLOUDBEES, 2026

“Model providers are heavily subsidized, and the models themselves are built on shifting sands. Every new model release follows the same pattern of high benchmarks, followed by hype, followed by the reality of usage and everyone complaining of them being ‘nerfed’ and burning through 2x-3x as many tokens to get the same job done.” ²⁹

— **Agentic Coding is a Trap,**
LARS FAYE, 2026

*“More concerning, developers who use AI at work report that they feel like they are de-skilling themselves and **losing their ability to do their jobs as well as they used to.***

There’s no way to evaluate whether that much code is well-written or secure — especially when hundreds of other programmers in the company are doing the same.

We’re building a rat’s nest of tech debt that will be impossible to untangle when these models become prohibitively expensive (any minute now...). ⁴²

— Software Developers Say AI Is Rotting Their Brains,
E. MAIBERG, 2026

⁴² [Software Developers Say AI Is Rotting Their Brains](#) ↗ - E. MAIBERG, MAY 2026

4. AI as a Software Engineering Tool

- LLMs should not be seen as a substitute for software engineering.

Software engineering is an unusual discipline, and a lot of people mistake it for being the same as typing code into an IDE.

... Because software engineering is all about finding what those pieces are, and how they should behave, and then how you can put them together to create a bigger piece, and then how you can put them together to create a bigger piece. And if we do that well, then in 10 years' time, we could have software that is far more capable than anything we could even imagine today." ⁴³

— Jeremy Howard,
THE DANGEROUS ILLUSION OF AI CODING?, 2026

⁴³ [The Dangerous Illusion of AI Coding?](#) ↗ - JEREMY HOWARD, 2026

“In ProgramBench, given only a program and its documentation, agents must architect and implement a codebase that matches the reference.

*... Our 200 tasks range from compact CLI tools to widely used software such as FFmpeg, SQLite, and the PHP interpreter. We evaluate 9 LMs and find that **none fully resolve any task.***

*... Models favor monolithic, single-file implementations **that diverge sharply from human-written code.**”⁴⁴*

— ProgramBench: Can Language Models Rebuild Programs From Scratch?,
YANG ET AL., 2026

⁴⁴ [ProgramBench: Can Language Models Rebuild Programs From Scratch?](#)  - YANG ET AL., ARXIV, MAY 2026

“One of the lesser spoken truths of working productively with LLMs as a software engineer on non-toy-projects is that it’s difficult. There’s a lot of depth to understanding how to use the tools, there are plenty of traps to avoid, and the pace at which they can churn out working code raises the bar for what the human participant can and should be contributing.

*... Iterating with coding agents to **produce production-quality** code that I’m confident **I can maintain in the future** feels like a different process entirely.”⁴⁵*

— Simon Willison,
VIBE ENGINEERING

⁴⁵ [Vibe engineering](#)  - SIMON WILLISON, 2025

*“I’m not very happy with the code quality and I think **agents bloat abstractions**, have poor code aesthetics, are very prone to copy pasting code blocks and **it’s a mess**, but at this point I stopped fighting it too hard and just moved on.”*⁴⁶

— Andrej Karpathy

⁴⁶ [Andrej Karpathy, 2026](#) ↗

“Programming, like writing, is an activity, where one iteratively sharpens what they’re doing as they do it.

*... But, **vibe coding** gives the illusion that your vibes are precise **abstractions**. They will feel this way right up until they leak, which will happen when you add enough features or get enough scale. **Unexpected behaviors (bugs) that emerge from lower levels of abstraction that you don’t understand** will sneak up on you and wreck your whole day.”*⁴⁷

— Reports of code's death are greatly exaggerated,
STEVE KROUSE

⁴⁷ Reports of code's death are greatly exaggerated ↗ - STEVE KROUSE, 2026

*“LLMs have effectively killed the cost of generating lines of code, but they **haven’t touched the cost of truly understanding a problem**. We’re seeing a flood of “apps built in a weekend,” but most of these are just thin wrappers around third-party APIs. They look impressive in a Twitter demo, but they often crumble the moment they hit the friction of the real world...*

*... In this new reality, **engineering expertise remains incredibly valuable**, but the nature of the role is shifting. Relevance is not fading. Instead, it is about leveraging these tools to build at a higher level than was previously possible. True expertise is now required to steer these systems and provide the technical oversight that LLMs currently lack.” ⁴⁸*

— **Code Is Cheap Now. Software Isn't,**
CHRIS GREGORI

⁴⁸ [Code Is Cheap Now. Software Isn't](#)  - CHRIS GREGORI, 2026

“Creating durable production code, managing a complex project, or crafting something truly novel still requires experience, patience, and skill beyond what today’s AI agents can provide on their own.

*... veteran software developers probably shouldn’t fear losing their jobs to these tools any time soon. **In fact, they may become busier than ever.***

... I don’t think AI tools will make human software designers obsolete. Instead, they may well help those designers become more capable.”⁴⁹

— Benj Edwards,
ARS TECHNICA

⁴⁹ [10 things I learned from burning myself out with AI coding agents](#) ↗ - BENJ EDWARDS, ARS TECHNICA, 2026

- **AI systems complement human expertise. They do not substitute for it.**

*“What the models do, the capabilities they have, and the quality of the content they produce is, to a large extent, **a reflection of the user.**”*⁵⁰

— **The skeptic's guide to generative AI assisted coding,**
ROB PATRO

⁵⁰ [The skeptic's guide to generative AI assisted coding](#)  - ROB PATRO, COMBINE-LAB, 2026

*“Experienced human software developers bring judgment, creativity, and domain knowledge that **AI models lack**. They know how to architect systems for long-term maintainability, how to balance technical debt against feature velocity, and when to push back when requirements don’t make sense.*

*... They can automate many tasks, but **managing the overarching project scope still falls to the person telling the tool what to do.**” ⁴⁹*

— Benj Edwards,
ARS TECHNICA, 2026

*“Knowing something about how good software development works helps a lot when guiding an **AI coding agent**—the tool amplifies your existing knowledge rather than replacing it.”* ⁴⁹

— Benj Edwards, ARS TECHNICA

*“**AI tools amplify existing expertise.** The more skills and experience you have as a software engineer the faster and better the results you can get from working with LLMs and coding agents.”* ⁴⁵

— Simon Willison, VIBE ENGINEERING

*“Relying too much on AI risks missing subtle bugs, architectural flaws, and vulnerabilities that only skilled engineers can catch. **Human oversight, critical thinking, and domain knowledge are indispensable** for both correcting errors and driving innovation as technology progresses.*

*Generative AI currently acts as seniority-biased technological change: **It disproportionately amplifies engineers who already possess systems judgment**, like a taste for architecture, debugging under uncertainty, and operational intuition.”¹⁵*

— **Redefining the Software Engineering Profession for AI**,
COMMUNICATIONS OF THE ACM, 2026

*“LLMs have evolved into enormously powerful engines that can produce vast amounts of artifacts. However, **they need to be guided to produce output that is actually valuable**. Without this, they can produce a mountain of garbage just as easily as gold.*

*... The fluency of the AI makes it easy to think you should interact with it like you would a junior engineer. **The best output comes from realizing that it is a machine that produces code and executes tasks**, and you should treat it as such.”⁵¹*

— 'AI makes me 10x productive', 'AI produces garbage',

MICHAEL ROTHROCK, 2026

⁵¹ 'AI makes me 10x productive', 'AI produces garbage' [↗](#) - M. ROTHROCK, 2026

*“When I was working on something I already understood deeply, **AI was excellent**. I could review its output instantly, catch mistakes before they landed and move at a pace I’d never have managed alone.*

*... When I was working on something I could describe but didn’t yet know, **AI was good but required more care**.*

*... When I was working on something where I didn’t even know what I wanted, **AI was somewhere between unhelpful and harmful**.*

*... But expertise alone isn’t enough. Even when I understood a problem deeply, **AI still struggled if the task had no objectively checkable answer**⁵²*

— Eight years of wanting, three months of building with AI,
L. MAGANTI, 2026

⁵² Eight years of wanting, three months of building with AI ↗ - L. MAGANTI, 2026

“LLMs optimize for plausibility over correctness. In this case, plausible is about 20,000 times slower than correct.

*... THIS is the failure mode. Not broken syntax or missing semicolons. The code is syntactically and semantically correct. **It does what was asked for. It just does not do what the situation requires.**”⁵³*

— Your LLM Doesn't Write Correct Code. It Writes Plausible Code.,
HŌRŌSHI, 2026

⁵³ [Your LLM Doesn't Write Correct Code. It Writes Plausible Code.](#) ↗ - HŌRŌSHI, 2026

5. Engineering Practices in the Age of AI

LLMs reward existing software engineering practices. The following list presents a concise set of practices through which AI systems can enhance the software development process. This list has been compiled from engineers' public notes, discussions with colleagues, and personal experience.

- **Code Review.** LLMs rely on pattern matching and memorization. They can be very effective at finding common issues, faulty logic, or missing assumptions in code. *This is not a substitute for human code review.*
- **Documentation.** Writing documentation is essential for understanding and maintaining software, but it is often a time-consuming and boring task. LLMs can help write documentation, or a draft, quickly, *to review and refine later.*
- **Code Explanation.** LLMs can parse code much faster than humans. As a result, they can be very useful for explaining code in context or for understanding the organization and workflow of large codebases.

- **Prototyping.** Productization is the most demanding phase of the development process, and it requires a clear idea of what to achieve, which is often not the case. LLMs can be very effective at quickly implementing new features or ideas, *even if the quality is far from production-ready.*
- **Discussing Ideas and Intuitions.** It is common to reflect on new ideas or revisit earlier ones when thinking through specific aspects of a problem. LLMs can discuss these topics much as a coworker might, even before prototyping.
- **Evaluating Alternatives.** There are dozens of ways to solve a given problem. LLMs can suggest alternatives that users can evaluate and then select the most suitable option *depending on the context.*

- **Enforcing Coding Style.** Large codebases involving several engineers tend to develop high-level practices that are often difficult to enforce with common tools. Coding style can fall into this category. Requirements such as “the code shall not use lambda expressions” are difficult to enforce with traditional tools but trivial for LLMs.
- **Testing.** Robust and comprehensive test suites prevent users *and LLMs* from unintentionally introducing bugs. Additionally, LLMs can quickly draft tests that can be adjusted later.
- **Refactoring.** Technology evolves faster than engineers can keep up with, especially in projects with limited engineering resources. LLMs are an excellent tool for modernizing codebases, but it is important to pay attention to *maintaining the underlying logic*.

- **Debugging.** LLMs can automate the debugging process, especially for simple bugs. They can compile, execute, analyze the output, or even navigate the git history and find connections in the code to identify problems or formulate hypotheses.
- **Vibe Coding Tools.** It is common to face limitations with open-source tools because each project has its own specific context. Engineering work often does not leave time to contribute to external projects to overcome these limitations. LLMs can implement small features while engineers stay focused on the actual code. One example could be adding a new check to clang-tidy.
- **Help with Unfamiliar Tasks.** It is common to work on projects outside area of expertise. For example, LLMs can assist by explaining new syntax and translating between languages.

- **Git Interaction.** LLMs show excellent capabilities when working with `git`. They can navigate history, reverse changes, and identify the root causes of bugs. They can also handle complex rebases without human intervention.